

Arquivos Binários

ICP141 - Programação de Computadores II

Prof. Marcos Tomazzoli Leipnitz
marcosleipnitz@ic.ufrj.br

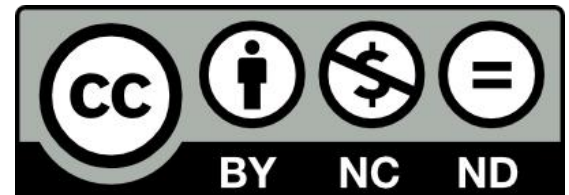


UFRJ

UNIVERSIDADE FEDERAL
DO RIO DE JANEIRO



INSTITUTO DE
COMPUTAÇÃO
UFRJ



- Arquivos em C
 - Arquivos binários
 - Funções de manipulação de arquivos binários
- Exemplos

Revisão: Arquivos em C

- Um arquivo em C nada mais é do que uma **sequência de bytes** (*streams*) armazenados em dispositivos de memória auxiliar de forma permanente
- Uso de periféricos que aceitam tanto a leitura como a gravação de dados (discos magnéticos, pen drives, etc).
- Toda a entrada ou saída de dados é processada através da manipulação de streams, **independentemente** do periférico utilizado (*device independent*).
- As operações de entrada e saída de dados de um arquivo são apoiadas pelo armazenamento temporário destes streams em áreas da memória principal denominadas **buffers**.

sequência de bytes (stream)



Arquivos Binários

- Agregado sequencial de bytes.
- **Logicamente**, o stream pode ser visto como um conjunto de dados simples ou estruturados
 - Ou seja, os bytes representam dados de tipos **primitivos** ou **estruturados** e podemos manipular as informações em função destes tipos.
 - O programador define como os bytes são tratados no programa.
- **Não tem limitação** teórica de tamanho.
- Comprimento do arquivo: número de elementos que apresenta no momento.
- Somente **um elemento** é acessível a **cada momento**.
 - Existe a noção de elemento corrente (atual).
 - Certas funções (fread, fwrite) alteram o elemento corrente.
- Acesso aos dados: **sequencial** ou **randômico**.



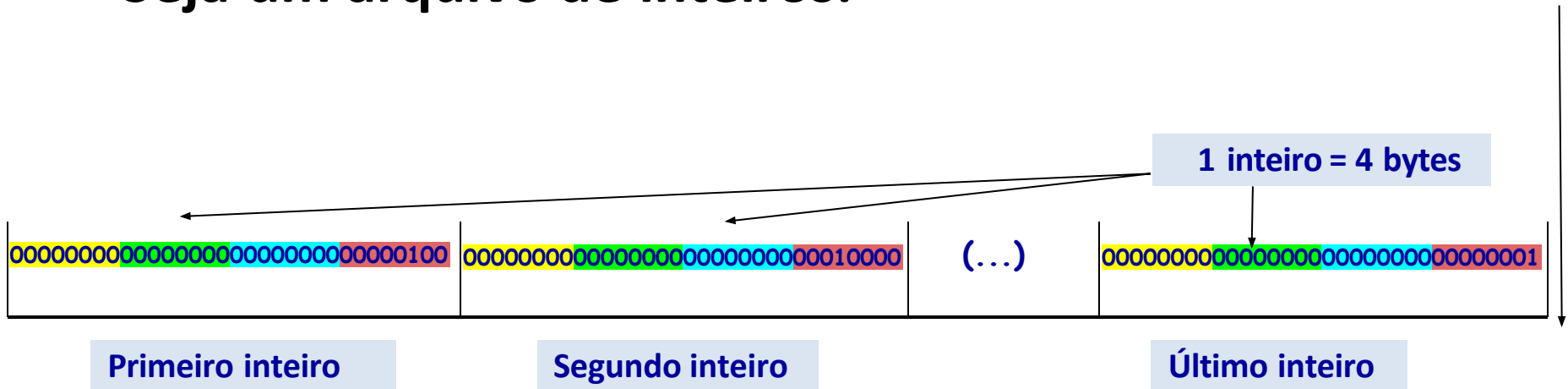
```
6D 00 69 00 6E 00 69 00 73 00 74 00 72 00 61 00
74 00 6F 00 72 00 73 00 41 00 64 00 6D 00 69 00
6E 00 69 00 73 00 74 00 72 00 61 00 74 00 6F 00
72 00 73 00 20 00 68 00 61 00 76 00 65 00 20 00
63 00 6F 00 6D 00 70 00 6C 00 65 00 74 00 65 00
20 00 61 00 6E 00 64 00 20 00 75 00 6E 00 72 00
65 00 73 00 74 00 72 00 69 00 63 00 74 00 65 00
64 00 20 00 61 00 63 00 63 00 65 00 73 00 73 00
20 00 74 00 6F 00 20 00 74 00 68 00 65 00 20 00
63 00 6F 00 6D 00 70 00 75 00 74 00 65 00 72 00
2F 00 64 00 6F 00 6D 00 61 00 69 00 6E 00 00 00
01 05 00 00 00 00 00 05 15 00 00 00 E4 DF E9 2C
A6 1B 05 04 56 F7 9B E5 F4 01 00 00 01 05 00 00
00 00 00 05 15 00 00 00 E4 DF E9 2C A6 1B 05 04
56 F7 9B E5 E8 03 00 00
```

O que está armazenado aqui?
Inteiros? Floats? Caracteres?
Apenas quem armazenou pode
saber.

Arquivos Binários

fim de
arquivo

Seja um arquivo de inteiros:



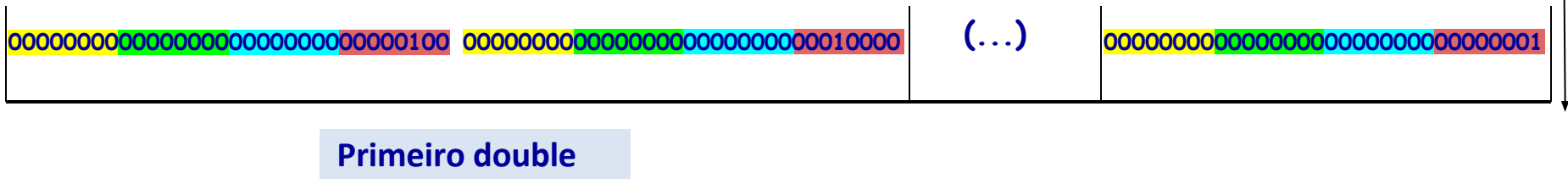
As separações entre *bytes* são apenas **ilustrativas**, no disco não existem!

Arquivos Binários

fim de
arquivo

Seja um arquivo de doubles:

1 double = 8 bytes



É o **programador** que define como as informações binárias são interpretadas em valores para serem utilizados no programa.

Manipulação de Arquivos Binários

Função	Funcionalidade
fopen	Abre/cria/recria arquivo
fclose	Fecha stream e descarrega conteúdos do buffer para o arquivo
fflush	Descarrega o conteúdo do buffer para o arquivo sem fechar o stream
fread	Lê dados do arquivo
fwrite	Grava dados em arquivo
feof	Indica se um arquivo chegou ao fim para leitura
ferror	Indica se a última operação com um arquivo produziu um erro
rewind	Coloca a posição corrente do arquivo para o início do arquivo

- O **sucesso** ou **fracasso** da execução destas funções pode ser verificado a partir do **valor retornado**.
- É altamente recomendado que os programas incluam esta verificação!

Modos de Abertura

Modo de abertura	Arquivo Existe	Arquivo não existe	Permissão de leitura	Permissão de escrita
rb	ok	erro	x	
wb	cria novo, antigo eliminado	cria		x
r+b ou rb+	ok	erro	x	x
w+b ou wb+	cria novo, antigo eliminado	cria	x	x
ab	novos dados acrescentados ao final	cria		x
a+b ou ab+	novos dados acrescentados ao final	cria	x	x

Os b's significam binário

Igual aos arquivos texto, mas com o b!

```
fread (&varbuffer, numbytes, quant, point_arq)
```

- Lê dados do arquivo para a **memória principal**.
 - Se valor retornado for menor que **quant**, em geral é indicação de **erro**!
 - É aconselhável **SEMPRE** testar o valor retornado pela função!
-
- Lê o dado a partir do elemento corrente e o armazena no endereço **&varbuffer**.
 - **numbytes** é o tamanho da unidade a ser lida em bytes (usar **sizeof**).
 - **quant** indica quantas unidades de **numbytes** serão lidas: logo, o número total de *bytes* que poderão ser lidos é igual a **numbytes*quant**.
 - **varbuffer** é um ponteiro para uma região da memória que armazenará os dados lidos.
 - A quantidade de **numbytes** efetivamente lida é **retornada** pela função, podendo ser **menor** do que o esperado se o **final do arquivo** for alcançado previamente ou **se ocorrer erro**.
 - Altera o **elemento corrente**.

```
fwrite (&varbuffer, numbytes, quant, point_arq )
```

- **Grava dados** no arquivo (buffer na memória principal).
 - Se valor retornado for menor que **quant**, em geral é indicação de **erro**!
 - É aconselhável **SEMPRE** testar o valor retornado pela função!
-
- O funcionamento é **similar à função de leitura**, só que para escrita.
 - Neste caso **&varbuffer** vai indicar o endereço de memória que contém os dados que queremos escrever no arquivo.
 - A quantidade **quant** efetivamente escrita (podem ocorrer erros ou buffer estar incompleto) é **retornada** pela função.
 - Altera o **elemento corrente**.

Atenção!

- O **modo de abertura** do arquivo deve ser compatível com as **funções** que serão utilizadas sobre o arquivo.
- **Abrir** o arquivo para **escrita** e utilizar funções para **leitura** resultará em **falha**.
- Do mesmo modo, **abrir** o arquivo para **leitura** e utilizar funções de **escrita** resultará em **falha**.
- Lembre-se de que **cada função** tem seu próprio modo de sinalizar a ocorrência de falha.

Exemplo 1

Faça um programa para obter 5 números inteiros do usuário (via teclado), e gravar em um arquivo binário. Depois, ler os valores do arquivo e armazenar em um vetor.



Solução

```
// Programa de manipulação de arquivos: escrita e leitura
#include <stdio.h>
#include <string.h>
#define NUMEL 5
int main( )
{
    FILE *arq; //declaração de um ponteiro para arquivo
    char nome[16];
    int vetor[NUMEL];
    int cont, valor;
    printf("Informe o nome do arquivo: ");
    scanf("%s", nome); // não aceita espaços
    // escrita do vetor
    if(!(arq = fopen(nome,"wb")))// abre para escrita e testa a abertura
        printf("Erro na abertura\n");
    else{
        printf("Entre %d valores inteiros\n", NUMEL);
        for (cont = 0; cont < NUMEL; cont++){
            scanf("%d", &valor);
            // escreve e testa a escrita
            if (fwrite(&valor, sizeof(int), 1, arq) != 1)
                printf("Erro na escrita!\n");
        }
        fclose(arq);
    } // continua...
```

```
// leitura do vetor, elemento a elemento

if(!(arq = fopen(nome,"rb"))) // abre para leitura
    printf("Erro na abertura\n");
else
{
    for (cont = 0; cont < NUMEL; cont++) // varre o arquivo
    {
        // le do arquivo e testa leitura
        if (fread(&vetor[cont], sizeof(int), 1, arq) == 1)
            printf("vetor[%d] = %d\n", cont, vetor[cont]);
        else
            printf("Erro na leitura!!\n");
    }
    fclose(arq);
}

return 0;
}
```

Versão com leitura de elementos **individuais**

```
// leitura do vetor de uma vez só
```

```
int result_leitura; //variavel para verificar se leitura deu certo
if(!(arq = fopen(nome,"rb"))) // abre para leitura e testa leitura
    printf("Erro na abertura\n");
else
{
    result_leitura = fread(vetor, sizeof(int), NUMEL, arq);
    fclose(arq);
}
if (result_leitura != NUMEL) // teste da leitura
    printf("Erro na leitura\n");
else
    for (cont = 0; cont < NUMEL; cont++)
        printf("vetor[%d] = %d\n", cont, vetor[cont]);

return 0;
}
```

Versão com leitura do vetor **inteiro**

Exercício 2

- A. Faça um programa para gravar um arquivo composto por estruturas que contenham nome, idade e altura de atletas.**
- B. Faça outro programa, agora para exibir as informações de um arquivo com estas estruturas.
- C. Faça um terceiro programa, para buscar no arquivo e imprimir a altura de um determinado atleta, cujo nome foi lido via teclado. O programa deve realizar essa operação continuamente até o usuário pedir para parar.
- D. Faça outro programa que insere um novo registro no final do arquivo.

Solução Item A

```
// Programa de manipulação de arquivos:
#include <stdio.h> //e demais biblioteca necessárias
#include <string.h>

typedef struct
{ /*declaração global da estrutura utilizada*/
    char nome[50];
    int idade;
    float altura;
}ATLETA;

int main( ){
    ATLETA buffer;
    FILE *arq; //declaração do ponteiro para arquivo
    char nome[16];
    int op;
    printf("Nome do arquivo: ");
    gets(nome); // Como seria com fgets?
    //cria ou grava por cima, validando
    if(!(arq = fopen(nome,"wb")))
        printf("Erro de criação de arquivo");
    else //coleta dados do usuário e grava no arquivo
    {
        ...
    }
}
```

Solução Item A

```
do{
    printf("\nNome: ");
    gets(buffer.nome);
    printf("\nIdade: ");
    scanf("%d",&buffer.idade);
    printf("\nAltura: ");
    scanf("%f",&buffer.altura);
    if (fwrite(&buffer, sizeof(ATLETA), 1, arq) != 1)
        printf("Erro de escrita\n");
    printf("\n1-InserirNovo, 2-Encerrar\n");
    scanf("%d",&op);
    getchar();
}while(op != 2);
fclose(arq); // fecha
}
```

Determina o número de bytes necessários para representar **ATLETA**.

```
return 0;
}
```

Exercício 2

- A. Faça um programa para gravar um arquivo composto por estruturas que contenham nome, idade e altura de atletas.
- B. Faça outro programa, agora para exibir as informações de um arquivo com estas estruturas.**
- C. Faça um terceiro programa, para buscar no arquivo e imprimir a altura de um determinado atleta, cujo nome foi lido via teclado. O programa deve realizar essa operação continuamente até o usuário pedir para parar.
- D. Faça outro programa que insere um novo registro no final do arquivo.

Solução Item B

```
//Utiliza as mesmas declarações do anterior
int main( ) {
    ATLETA buffer;
    char nome[16];
    FILE *arq; //declaração de ponteiro para arquivo

    printf("Nome do arquivo: ");
    scanf("%s", &nome); // não aceita espaços
    if(!(arq = fopen(nome,"rb"))) // abre para leitura, posiciona no início
        printf("Erro na abertura\n");
    else // listagem sequencial
    {
        printf("-----Comeco da listagem-----\n");
        while(!feof(arq)) // enquanto não chegou no final
        {
            if(fread(&buffer, sizeof(ATLETA), 1 ,arq) == 1)
            {
                printf("Nome: %s\n",buffer.nome);
                printf("Idade: %d\n",buffer.idade);
                printf("Altura: %.2f\n\n",buffer.altura);
            }
        }
        printf("-----Fim da listagem-----\n");
        fclose(arq);
    }
}
```

Exercício 2

- A. Faça um programa para gravar um arquivo composto por estruturas que contenham nome, idade e altura de atletas.
- B. Faça outro programa, agora para exibir as informações de um arquivo com estas estruturas.
- C. Faça um terceiro programa para buscar no arquivo e imprimir a altura de um determinado atleta, cujo nome foi lido via teclado. O programa deve realizar essa operação continuamente até o usuário pedir para parar.**
- D. Faça outro programa que insere um novo registro no final do arquivo.

Solução Item C

Como resolver esse problema?

Quais os cuidados que devemos ter neste caso?



Solução Item C

Como resolver esse problema?

Quais os cuidados que devemos ter neste caso?

É preciso garantir que cada busca **sempre comece a partir do início** do arquivo. Como?



Solução Item C

```
//mesmas inclusões e definições do programa anterior
int main()
{
    ATLETA buffer;
    char nome[16];
    FILE *arq; //declaração de ponteiro para arquivo
    int encontrado, op;
    char procurado[31];

    printf("Nome do arquivo: ");
    scanf("%s", &nome); // não aceita espaços
    if(!(arq = fopen(nome,"rb"))) //leitura, validando
        printf("Erro abertura");
    else
        do
        {
            /*reposiciona registro corrente no início do arquivo.
            Necessario se mais de uma busca for realizada*/
            rewind(arq);
            encontrado = 0; //indica não encontrado (ainda)
            getchar();//limpa o buffer
            printf("Nome do atleta procurado: ");
            gets(procurado); // Como seria com fgets?
            ...
        }
```

Solução Item C

```
...
//para quando acha ou no fim do arquivo
while(!feof(arq) && encontrado == 0)
    if(fread(&buffer,sizeof(ATLETA),1,arq) == 1)
        if(!(strcmp(buffer.nome,procurado))) //se strings são
            iguais (==0)
        {
            printf("\nAltura de %s:
%.2fm",procurado,buffer.altura);
            encontrado = 1; // indica que encontrou
        }
    if(!encontrado)
        printf("\nAtleta nao foi localizado.");
    printf("\n1-BuscarOutro, 2-Encerrar\n");
    scanf("%d",&op);
}while(op != 2);
fclose(arq);
return 0;
}
```

Exercício 2

- A. Faça um programa para gravar um arquivo composto por estruturas que contenham nome, idade e altura de atletas.
- B. Faça outro programa, agora para exibir as informações de um arquivo com estas estruturas.
- C. Faça um terceiro programa para buscar no arquivo e imprimir a altura de um determinado atleta, cujo nome foi lido via teclado. O programa deve realizar essa operação continuamente até o usuário pedir para parar.
- D. Faça outro programa que insere um novo registro no final do arquivo.**

Solução Item D

Como resolver esse problema?
Como devemos abrir o arquivo neste caso?



Solução Item D

```
int main( ) {
    ATLETA buffer; FILE *arq;
    char nome[16];
    int op;
    printf("Nome do arquivo: ");
    scanf("%s",&nome);
    if(!(arq = fopen(nome,"ab"))) //abre para acrescentar dados
        printf("Erro na abertura para acrescentar");
    else
    {
        do{ // gravados a partir do final do arquivo:
            getchar();//limpa o buffer
            printf("\nNome: ");
            gets(buffer.nome);
            printf("\nIdade: ");
            scanf("%d",&buffer.idade);
            printf("\nAltura: ");
            scanf("%f",&buffer.altura);
            if (fwrite(&buffer,sizeof(ATLETA),1,arq) != 1)
                printf("Erro na escrita\n");
            printf("\n1-Inserir novo, 2-Encerrar\n");
            scanf("%d", &op);
        }while(op != 2);
        fclose(arq);
    }
    return 0;
}
```

Modo de abertura para
acrescentar dados ao **final**
do arquivo.

Cuidado com o feof!

Programa escreve N números inteiros em um arquivo, depois lê os números do arquivo.

```
#include <stdio.h>
#define NOME_ARQ "arquivo.bin"
#define N 100

int main( ) {
    FILE *arq;
    int i;
    if(!(arq = fopen(NOME_ARQ,"wb"))){
        printf("Erro na abertura\n");
    }else{// abertura realizada com sucesso
        printf("Realizando escrita de dados\n");
        for(i=0;i<N;i++){
            if(fwrite(&i,sizeof(int),1,arq)==1){
                printf("%d gravado com sucesso\n",i);
            }else{
                printf("Erro na gravação\n");
            }
        }
        fclose(arq);
    }
```

```
if(!(arq = fopen(NOME_ARQ,"rb"))){
    printf("Erro na abertura\n");
}else{// abertura realizada com sucesso
    printf("Realizando leitura de dados\n");
    while(!feof(arq)){/* enquanto não chegou
        no final, testa leitura e desconsidera
        a última */
        if(fread(&i, sizeof(int), 1 ,arq)==1){
            printf("Lido: %d\n",i);
        }else{
            printf("Erro na leitura");
        }
    }
    fclose(arq);
}
```

Cuidado com o feof!

Programa escreve N números inteiros em um arquivo, depois lê os números do arquivo.

```
#include <stdio.h>
#define NOME_ARQ "arquivo.bin"
#define N 100

int main( ) {
    FILE *arq;
    int i;
    if(!(arq = fopen(NOME_ARQ,"wb"))){
        printf("Erro na abertura\n");
    }else{// abertura realizada com sucesso
        printf("Realizando escrita de dados\n");
        for(i=0;i<N;i++){
            if(fwrite(&i,sizeof(int),1,arq)==1){
                printf("%d gravado com sucesso\n",i);
            }else{
                printf("Erro na gravação\n");
            }
        }
        fclose(arq);
    }
```

```
if(!(arq = fopen(NOME_ARQ,"rb"))){
    printf("Erro na abertura\n");
}else{// abertura realizada com sucesso
    printf("Realizando leitura de dados\n");
    while(!feof(arq)){/* enquanto não chegou
        no final, testa leitura e desconsidera
        a última */
        if(fread(&i, sizeof(int), 1 ,arq)==1){
            printf("Lido: %d\n",i);
        }else{
            printf("Erro na leitura");
        }
    }
    fclose(arq);
}
```

- Essa mensagem vai ser exibida uma vez no final, porque o comando feof só retorna verdadeiro **após ocorrer uma falha na leitura**.
- Por isso é importante verificar se o fread foi executado com **sucesso**.
- Caso tenha ocorrido falha, deve-se **desconsiderar o valor lido**, pois se trata de um **valor inválido**.

Cuidado com o feof!

Programa escreve N números inteiros em um arquivo, depois lê os números do arquivo.

```
#include <stdio.h>
#define NOME_ARQ "arquivo.bin"
#define N 100

int main( ) {
    FILE *arq;
    int i;
    if(!(arq = fopen(NOME_ARQ,"wb"))){
        printf("Erro na abertura\n");
    }else{// abertura realizada com sucesso
        printf("Realizando escrita de dados\n");
        for(i=0;i<N;i++){
            if(fwrite(&i,sizeof(int),1,arq)==1){
                printf("%d gravado com sucesso\n",i);
            }else{
                printf("Erro na gravação\n");
            }
        }
        fclose(arq);
    }
```

```
if(!(arq = fopen(NOME_ARQ,"rb"))){
    printf("Erro na abertura\n");
}else{// abertura realizada com sucesso
    printf("Realizando leitura de dados\n");
    while(!feof(arq)){/* enquanto não chegou
        no final, testa leitura e desconsidera
        a última */
        if(fread(&i, sizeof(int), 1 ,arq)==1){
            printf("Lido: %d\n",i);
        }else{
            printf("Erro na leitura");
        }
    }
    fclose(arq);
}
```

- Este trecho funciona independentemente de quantos valores inteiros estão armazenados no arquivo.
- O feof é útil para lidar com casos em que **não sabemos** quantos elementos estão armazenados no vetor.

Gravação e Leitura de Arranjos

```
#include <stdio.h>
#define N 3

int main(){
    FILE *arquivo;
    int i,j;
    int matriz2[N][N];
    int matriz[N][N] = {1,2,3,4,5,6,7,8,9};

    arquivo = fopen("teste.dat","wb");

    if(arquivo != NULL){
        if(fwrite(matriz,sizeof(matriz),1,arquivo)
           != 1){
            printf("Problemas na escrita\n");
        }else{
            printf("Gravado\n");
            fclose(arquivo);
        }
    }else{
        printf("Problemas na abertura\n");
    }
}
```

```
arquivo = fopen("teste.dat","rb");
if(arquivo != NULL){
    if(fread(matriz2,sizeof(matriz2),1,arquivo)
       != 1){
        printf("Problemas na escrita\n");
    }else{
        printf("Elementos\n");
        for(i=0;i<N;i++){
            for(j=0;j<N;j++){
                printf("%d\t",matriz2[i][j]);
            }
        }
        fclose(arquivo);
    }
}else{
    printf("Problemas na abertura\n");
}

return 0;
}
```

Quando aplicado em um arranjo, **sizeof** retorna o número de bytes necessários para representar **todo o arranjo**.

Cuidado com o sizeof!

```
#include <stdio.h>
```

```
void exhibeVetor(int v[]){  
    int i,tamanho;  
    tamanho = sizeof(v)/sizeof(int);  
    printf("Tamanho, dentro da função: %d\n",tamanho);  
    for(i=0;i<tamanho;i++){  
        printf("%d\t",v[i]);  
    }  
    printf("\n");  
}
```

```
int main(){  
    int v[5] = {5,7,8,4,2};  
    int tamanho = sizeof(v)/sizeof(int);  
    printf("Tamanho, na main: %d\n",tamanho);  
    exhibeVetor(v);  
    return 0;  
}
```

Qual é o resultado da execução?

Cuidado com o sizeof!

```
#include <stdio.h>
```

```
void exhibeVetor(int v[]){  
    int i,tamanho;  
    tamanho = sizeof(v)/sizeof(int);  
    printf("Tamanho, dentro da função: %d\n",tamanho);  
    for(i=0;i<tamanho;i++){  
        printf("%d\t",v[i]);  
    }  
    printf("\n");  
}
```

```
int main(){  
    int v[5] = {5,7,8,4,2};  
    int tamanho = sizeof(v)/sizeof(int);  
    printf("Tamanho, na main: %d\n",tamanho);  
    exhibeVetor(v);  
    return 0;  
}
```

Qual é o resultado da execução?

```
Tamanho, na main: 5  
Tamanho, dentro da função: 2  
5       7
```

- Dentro da função, v é visto como um ponteiro para inteiro.
- sizeof(v) retorna o tamanho em bytes de um ponteiro para inteiro.

- Descreve um conjunto de cadeias de caracteres, de forma concisa, sem precisar listar todos os elementos do conjunto:

[] Lista de caracteres

- Casa uma ocorrência de qualquer caractere contido na lista
 - Ex.: `[abc]` casa "a", "b" ou "c" ou " "
- É possível definir intervalos de caracteres
 - Ex.: `[a-z]` casa qualquer caractere de "a" a "z"; `[0123456789]` é igual a `[0-9]`; `[A-Z]` casa qualquer caracter de "A" a "Z".

[^] Lista negada de caracteres

- Casa uma ocorrência de qualquer caractere **não** contido na lista
 - Ex.: `[^abc]` casa qualquer caractere que não seja "a", "b" ou "c";
`[^a-z]` casa qualquer caractere que não esteja em minúsculo

Exemplo - fscanf()

```
#include <stdio.h>

int main()
{
    char nome[50];
    float media;
    int i;

    FILE *parq;
    parq=fopen("teste.txt","r");
    if(parq==NULL)
    {
        printf("Erro na abertura
do arquivo");
        return -1;
    }

    i=0;
    while (!feof(parq))
    {
        printf("---%do. aluno\n",i+1);
        fscanf(parq,"% [A-Z a-z] %f\n",
            nome, &media);
        printf("Nome: %s - Media:
%.1f\n",nome,media);

        i++;
    }
    fclose(parq);
    return 0;
}
```

[A-Z a-z] → Expressão regular!
Irá ler caracteres de todo alfabeto em
maiúsculo, o espaço em branco e todo
alfabeto em minúsculo, respectivamente.